Forward and Inverse Kinematics with Joint-Space Feedback for Keyboard Typing on a LeKiwi Mobile Manipulator

Ernest Saakian[1] Enkel Cerri[1] Edmond Wong[1]

***Abstract*— Given the joint positions of a robot, how do we know the end effector position? Given a target end effector position, how do we know which joint positions are needed to achieve it? Forward kinematics (FK) and inverse kinematics (IK) methods can answer these exact questions. FK is computed with a chain of frame transformations. IK is an optimization problem where we try to minimize the difference between our predicted end effector position using FK and the target position by adjusting the joint positions. We set up our system as multiple ROS 2 nodes to allow our controller to drive both a simulated and a real robot. Our FK and IK were initially validated using the simulator, and eventually we typed "pool" on a physical keyboard with and without feedback. With the arm still, the FK fingertip position matched the simulator's ground truth to within 0.011 mm on average. On the real robot, the elbow settled 2.1–2.4° from its command under gravity at every key press, and the mean key-press error was 11.6 mm with 0 of 4 keys correct. An integral correction on the joint angles reduced it to 2.7 mm with 3 of 3 keys correct, and the robot then typed "pool" with 4 of 4 keys correct.**

I. INTRODUCTION

When driving a robot, we have to specify the robot state in joint space where we list out the position of every single motor on the robot [1]. The joint space does not map clearly to the end effector position, which makes it both difficult to visualize where the robot end effector would end up and which motor positions would yield the desired end effector position.

Section II describes the kinematic model, the forward and inverse kinematics, and the typing controller with joint-space feedback. Section III details how the controller was validated in simulation and on the robot, and how the physical measurements were taken. Section IV presents the results, and Section V summarizes the performance of the system.

II. METHOD

A. System Overview

The robot is a LeKiwi mobile manipulator [7] (Fig. 1): a three-wheel omnidirectional base that carries an SO-101 arm [4] with five revolute joints (shoulder pan, shoulder lift, elbow flex, wrist flex and wrist roll) and a gripper. The six arm joints and the three wheels are driven by nine Feetech STS3215 position-controlled servos on one 12 V serial bus, which a Beelink mini PC on the base reads and commands through LeRobot [4]. A second SO-101 arm is the leader arm for teleoperation.

The authors are with the Tandon School of Engineering, New York University, Brooklyn, NY, USA. ROB-GY 6003, Team 12. es6379, ec5874, ew1697 @nyu.edu



Fig. 1. LeKiwi mobile manipulator: three-wheel omnidirectional base and SO-101 arm with five revolute joints and a gripper.

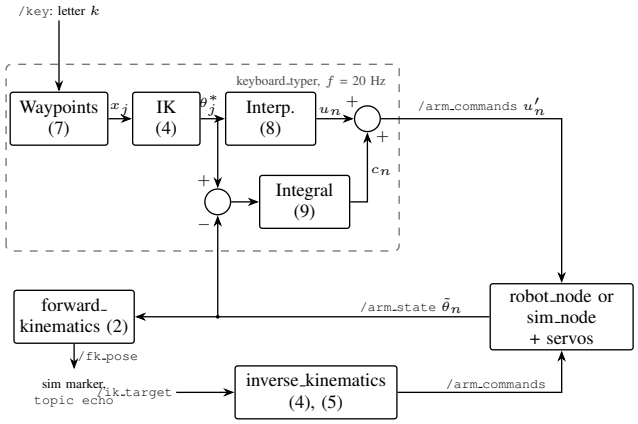


Fig. 2. Control block diagram and ROS 2 graph. Inside keyboard_typer (dashed box), a letter k on /key becomes four fingertip waypoints x_j (7); each is solved by IK (4), interpolated in joint space (8) and corrected by the integral term c_n (9)–(10), which acts only while a waypoint is held. robot_node (or sim_node) returns the measured angles $\tilde{\theta}_n$ on /arm_state, which close the loop and feed forward_kinematics. inverse_kinematics sends a single command per /ik_target.

All software runs as ROS 2 [2] (Jazzy) nodes on the mini PC (Fig. 2, Table I). Joint angles are exchanged in degrees, followed by the gripper opening in percent. robot_node applies each message on /arm_commands to the servos and publishes the measured angles on /arm_state at 30 Hz. sim_node runs the same arm in MuJoCo [3] at 50 Hz with the same two topics, so every node in Table I runs unchanged on the simulator or on the robot. forward_kinematics publishes the fingertip position on /fk_pose for every /arm_state message. inverse_kinematics sends the fingertip to a position received on /ik_target in a single command. keyboard_typer receives a letter or a word on /key and computes, interpolates and corrects the joint commands that press the keys at 20 Hz.

TABLE I

ROS 2 NODES AND TOPICS

Node	Subscribes	Publishes
robot_node	/arm_commands	/arm_state
sim_node	/arm_commands, /fk_pose	/arm_state, /true_fingertip, /key_pressed
leader_node	–	/arm_commands
forward_kinematics	/arm_state	/fk_pose
inverse_kinematics	/ik_target, /arm_state	/arm_commands
keyboard_typer	/key, /arm_state	/arm_commands

TABLE II

JOINT AXES AT THE ZERO POSE, IN THE BASE FRAME

Joint i	Point q_i (mm)	Axis $\hat{\omega}_i$
1 shoulder pan	(67.2, 0.0, 84.5)	(0, 0, −1)
2 shoulder lift	(97.8, −18.3, 136.7)	(0, 1, 0)
3 elbow flex	(115.8, −16.8, 251.2)	(0, 1, 0)
4 wrist flex	(248.8, −15.3, 280.0)	(0, 1, 0)
5 wrist roll	(307.8, 4.8, 295.8)	(−0.966, 0, −0.259)

B. Kinematic Model

All positions are expressed in the robot's base frame $\{b\}$, with its origin at the centre of the bottom base plate, x forward (towards the arm), y to the robot's left and z up. The joint angles $\theta_1, \ldots, \theta_5$ belong to shoulder pan, shoulder lift, elbow flex, wrist flex and wrist roll. The zero pose is $\theta = 0$, the middle of every joint's range. The end effector is the tip of the gripper's fixed finger; the gripper joint moves only the other jaw and is not part of the chain. At the zero pose the fingertip is at $p_h = (412.6, 4.8, 310.4)$ mm and the finger points along $d_h = (0.966, 0, 0.259)$, 15.0° above the x axis. Each joint i is described by a point q_i on its axis and the unit axis direction $\hat{\omega}_i$ at the zero pose (Table II). A positive θ_i turns the links after joint i counter-clockwise about $\hat{\omega}_i$, so $\theta_1 > 0$ turns the arm to the robot's right. The joint limits are $\theta_l = (-115, -100, -90, -100, -100)^\circ$ and $\theta_u = (115, 90, 95, 100, 160)^\circ$. These parameters were read from the course's MuJoCo model, which uses the geometry of the SO-100 arm, while the physical arm is an SO-101.

C. Forward Kinematics

Let $\theta \in \mathbb{R}^5$ be the joint angles. For joint i, $[\hat{\omega}_i]$ is the skew-symmetric matrix with $[\hat{\omega}_i]\, v = \hat{\omega}_i \times v$, and $R_i \in SO(3)$ is the rotation by θ_i about $\hat{\omega}_i$, given by Rodrigues' formula (1). The transform that rotates the links after joint i about the axis through q_i is the 4×4 matrix in (2). The product of these transforms from the base ($i = 1$) to the wrist ($i = 5$) is the product-of-exponentials formula [1], and applied to p_h it gives the fingertip position $p(\theta)$. The finger direction $d(\theta)$ follows from the rotations alone (3).

$$R_i = I_3 + \sin\theta_i\, [\hat{\omega}_i] + (1 - \cos\theta_i)\, [\hat{\omega}_i]^2 \tag{1}$$

$$\begin{bmatrix} p(\theta) \\ 1 \end{bmatrix} = \prod_{i=1}^{5} \begin{bmatrix} R_i & (I_3 - R_i)\, q_i \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_h \\ 1 \end{bmatrix} \tag{2}$$

$$d(\theta) = R_1 R_2 R_3 R_4 R_5\, d_h \tag{3}$$

Equation (2) is the chain of homogeneous transforms of our Assignment 02 code, written with every joint axis expressed in $\{b\}$ at the zero pose: R_i rotates about an arbitrary unit axis instead of a local x, y or z axis, and the points q_i replace the link offsets. The implementation applies the five rotations to p_h and d_h from $i = 5$ down to $i = 1$, which evaluates the same product from right to left. Over 1000 random joint vectors within the limits, it differed from an explicit product of 4×4 matrices by at most 1.8 μm, from the 5-digit rounding of $\hat{\omega}_5$. The forward_kinematics node evaluates (2) for every `/arm_state` message and publishes $p(\tilde{\theta})$ as a `PoseStamped` in frame `base` on `/fk_pose`. sim_node subscribes to `/fk_pose` and draws a magenta sphere of 8 mm radius at that point (Fig. 4).

D. Inverse Kinematics

Given a desired fingertip position p_d and finger direction d_d, IK solves the bounded optimization (4). Here θ_l and θ_u are the joint limits, $w = 0.05$ m weights the dimensionless direction error against the position error in metres, and $d_d = -e_3$ (finger straight down) for every target in this work, with $e_3 = (0, 0, 1)$. The solver is SciPy's [5] bounded quasi-Newton method L-BFGS-B [6] with finite-difference gradients and at most 500 iterations, started from θ_0: the current measured angles or the previous waypoint's solution. A solution θ^* is accepted only if (5) holds with $\epsilon = 5$ mm; otherwise the target is reported as unreachable and no command is sent.

$$\theta^* = \arg\min_{\theta_l \le \theta \le \theta_u} \|p_d - p(\theta)\| + w\, \|d_d - d(\theta)\| \tag{4}$$

$$\|p_d - p(\theta^*)\| \le \epsilon \tag{5}$$

Two properties of the model shape this problem. First, d_h is parallel to $\hat{\omega}_5$, so θ_5 does not change $d(\theta)$. Second, $\hat{\omega}_2 = \hat{\omega}_3 = \hat{\omega}_4$ and R_1 leaves e_3 unchanged, so the tilt φ of the finger from $-e_3$ depends on three joints only:

$$\varphi = |\theta_2 + \theta_3 + \theta_4 - 105^\circ| \tag{6}$$

The target therefore fixes four quantities (three position coordinates and $\varphi = 0$) with five joints. θ_5 still moves the fingertip, because p_h lies 13.0 mm from the wrist-roll axis, so the solver optimizes all five angles, and θ_0 selects one of the remaining solutions.

E. Keyboard Typing Controller

Let $x_k \in \mathbb{R}^3$ be the recorded position of the top of key k and x_H the recorded home position, both in $\{b\}$ (Section III-C). keyboard_typer accepts one letter or a word on `/key`; spaces are ignored, a message that contains an unmapped letter is rejected before the arm moves, and a message that arrives while a word is being typed is ignored. For each letter k it builds the four fingertip waypoints (7): above the key at hover height h, below the key top at press depth δ, above the key again, and home. Each waypoint $j = 1, \ldots, 4$ is solved with (4) for $p_d = x_j$ and $d_d = -e_3$, started from

the previous solution, which keeps consecutive joint targets θ_j^* close together. θ_0^* is the measured angle vector when the letter starts. If any waypoint fails (5), the letter and the rest of the word are not typed.

$$x_1 = x_k + h\,e_3, \quad x_2 = x_k - \delta\,e_3, \quad x_3 = x_1, \quad x_4 = x_H \tag{7}$$

The node runs at $f = 20$ Hz. At step n of move j, counted from the step at which move j starts, the command $u_n \in \mathbb{R}^5$ interpolates linearly in joint space from θ_{j-1}^* to θ_j^* over $N = T_m f$ steps (8), where T_m is the duration of one move. The waypoint is then held for $T_s f$ further steps before move $j+1$ starts. The gripper command is 0 % (closed) throughout. Because every letter ends at x_H, the single message "pool" produces the sequence p–home–o–home–o–home–l. One letter takes $4(T_m + T_s) = 15.2$ s on the robot, plus 0.37 s per IK solve on the mini PC.

$$u_n = \theta_{j-1}^* + \left(\theta_j^* - \theta_{j-1}^*\right) \min\!\left(1, \frac{n}{N}\right), \quad N = T_m f \tag{8}$$

F. Joint-Space Feedback Correction

Each STS3215 servo closes a proportional position loop internally; LeRobot sets its proportional coefficient to 16, half of the default 32, to reduce oscillation. Under a constant load torque, such as gravity on the links or the spring of a pressed key, a proportional loop settles where the torque from the position error balances the load, so the joint settles at an offset from its command in the direction of the load. Without correction, the elbow settled 2.1–2.4° from its command in the direction of gravity at every key press (Section IV).

keyboard_typer therefore adds an integral correction $c_n \in \mathbb{R}^5$ to the interpolated command. Let $\tilde{\theta}_n$ be the latest measured joint angles from `/arm_state`, K the integral gain per step, and sat_{c_m} the element-wise clipping to $[-c_m, c_m]$. While waypoint j is held ($N \le n \le N + T_s f$), the joint error is accumulated (9), and the published command is (10):

$$c_n = \begin{cases} \mathrm{sat}_{c_m}(c_{n-1} + K(\theta_j^* - \tilde{\theta}_n)), & N \le n \le N + T_s f \\ c_{n-1}, & n < N \end{cases} \tag{9}$$

$$u'_n = u_n + c_n \tag{10}$$

c is zero when the node starts and is not reset between waypoints or letters, so each move starts with the correction learned at the previous waypoint. In steady state $c_n = c_{n-1}$ requires $\tilde{\theta} = \theta_j^*$, so a constant load leaves no steady-state joint error as long as $|c| < c_m$. If the servo reached each new command within one step, the error would decay as $(1-K)^n$, to 7 % of its initial value after the $T_s f = 16$ steps of a hold. The limit $c_m = 4°$ per joint bounds the extra command, which limits how far the finger is pushed into the keyboard when a target cannot be reached and stops the integral from winding up. Table III lists the parameters.

TABLE III
CONTROLLER PARAMETERS

Symbol	Meaning	Robot	Sim.
w	direction weight in (4)	0.05 m	0.05 m
ϵ	IK acceptance threshold	5 mm	5 mm
h	hover height above key	40 mm	40 mm
δ	press depth below key top	5.5 mm†	8 mm
T_m	duration of one move	3.0 s	1.5 s
f	command rate of keyboard_typer	20 Hz	20 Hz
T_s	hold time at each waypoint	0.8 s	0.8 s
K	integral gain per step	0.15	0.15
c_m	correction limit per joint	4°	4°
–	robot_node / sim_node rate	30 Hz	50 Hz
–	robot_node step limit	15°	–

†2.5 mm and 4 mm in the two measured runs (Section III-E).

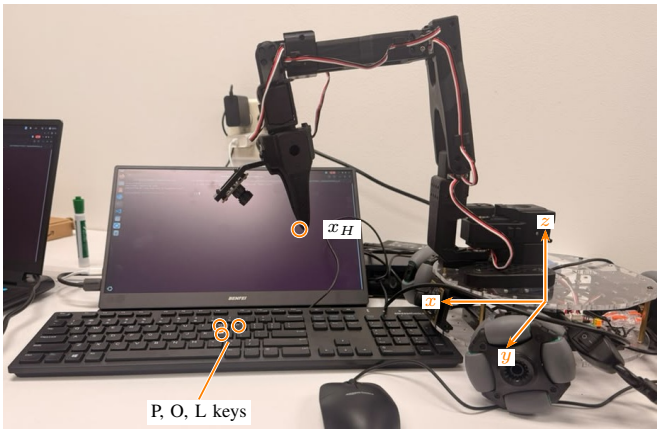


Fig. 3. Experimental setup. The LeKiwi stands to the right of the keyboard with the arm facing it and the fingertip at the home position x_H, 98.7 mm above the mean height of the recorded P, O and L key tops. The base-frame axes are drawn schematically at the robot's base; y points towards the camera.

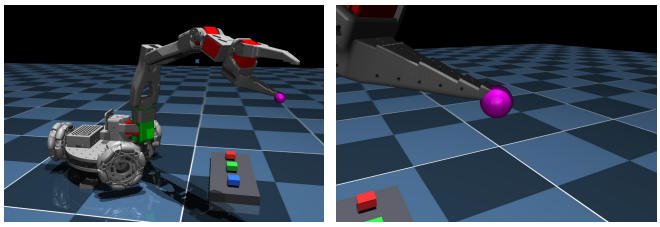
III. EXPERIMENT DESIGN

A. Hardware and Software Setup

Experimentation was done both on hardware and simulation. Our hardware setup was an onboard computer, an SO-101 follower arm [4] on a LeKiwi base [7], as well as the SO-101 leader arm. While only the follower arm and the keyboard were necessary, it was observed that recording positions of the keyboard keys was more accurately done with teleoperation with the leader arm instead of just moving the follower arm into place physically and then just recording the position (Section III-C). The SO-101 is the robot placed to the right of a full-size keyboard, with the arm facing it (Fig. 3). Which key registered was read from the characters typed on the computer the keyboard is connected to. For simulation we used MuJoCo [3]. Our MuJoCo setup involves an environment with a virtual keyboard with 3 keys, a, s and d, whose caps are 24 mm wide and 70 mm apart (a physical keyboard has a key pitch of 19.05 mm); they travel 6 mm and register at 3 mm (Fig. 4a).

B. Forward Kinematics Validation in Simulation

The simulator was started with the typing launch file (sim_node with the keyboard scene, forward_kinematics, inverse_kinematics and keyboard_typer), and the simulated arm was driven with the leader arm through leader_node



(a) (b)

Fig. 4. FK marker in the simulator for $\theta = (40, 20, -30, 50, 0)^\circ$. The magenta sphere (8 mm radius) is drawn at $p(\tilde{\theta})$ from (2), the point sim_node draws for `/fk_pose`; rendered offline with the simulator's own rendering and marker code. (a) Arm and the simulated keys a, s and d. (b) Close-up of the fixed finger; $p(\tilde{\theta})$ is 0.006 mm from the simulator's fingertip.

for 39.6 s. The marker drawn at `/fk_pose` was checked visually against the fingertip (Fig. 4). For a number, a logger subscribed to `/fk_pose` and to `/true_fingertip`, the position of the simulator's fingertip site in $\{b\}$, and paired each `/fk_pose` message with the `/true_fingertip` message closest in time, within 10 ms (sim_node publishes every 20 ms). For each pair it recorded the FK error (11), where p_{gt} is the simulator's fingertip and $\tilde{\theta}$ the joint angles of the same simulation step. Pairing by timestamp matters: in a scripted joint sweep, pairing the latest message of each topic instead produced a maximum error of 54 mm, against 2.7 mm with timestamp pairing.

$$e_{FK} = \|p(\tilde{\theta}) - p_{gt}\| \tag{11}$$

C. Keyboard Mapping

To map the keyboard keys, the robot was initially moved by hand to the position of each key; however, there was an issue with this. When moving the robot by hand, the robot's position also included the robot's flex as the hand dragged the robot towards the correct position; this movement was not related to the servos' positions, so it was not picked up. A better way to map the keyboard was to teleoperate the robot instead; this ensures that the robot has reached a position because its servos carried it there. With the motors on and robot_node at 30 Hz with a 15° step limit, the follower was driven with the leader arm until the fingertip rested on top of each key without pressing it, and the positions were recorded using `ros2 topic echo` on the topics `/fk_pose` and `/arm_state` (Table V). In addition we recorded a home position above the keyboard. The robot and keyboard were not moved between the mapping and the typing runs.

D. End-Effector Control

For the purposes of testing inverse kinematics, individual positions were published on `/ik_target`. The inverse_kinematics node solves (4) from the current `/arm_state` with $d_d = -e_3$ and sends the solution to `/arm_commands` in one message; if the error exceeds $\epsilon = 5$ mm (5), the target is treated as unreachable and refused. This test was run in the simulator only: three targets were published 6 s apart, $(280, 50, 50)$ mm, the home position $(266, 19, 90)$ mm and $(900, 0, 300)$ mm, which is outside the arm's reach, and the fingertip position was read from `/true_fingertip` after each move. For keyboard typing, the inverse kinematics code was used as a library and the robot was controlled by the keyboard_typer node (Section II-E).

E. Typing Experiments and Accuracy Measurement

Typing accuracy was measured with a logger node that subscribes to `/key`, `/arm_commands` and `/arm_state` without publishing. It evaluates (2) for every `/arm_state` message, detects a press as an interval in which the fingertip is more than 1 mm below the lowest key top, takes the sample n' with the lowest fingertip in each interval, and reports the horizontal press error (12), where $[I_2 \;\; 0]$ selects the x and y components, and the depth below the press target $x_k - \delta e_3$. e_p is computed from the measured joint angles through the model; it includes the servo errors but not the error of the model itself, for which no external measurement was available. Whether the intended key registered was read from the characters typed on the computer (robot) and from `/key_pressed` (simulator).

$$e_p = \left\| [I_2 \quad 0] \left(p(\tilde{\theta}_{n'}) - x_k + \delta\, e_3 \right) \right\| \tag{12}$$

Four conditions were measured, each with letters sent one at a time:

- Simulator, $K = 0$ and $K = 0.15$: keys a, s, d, with all other parameters as in Table III.
- Robot, $K = 0$: keys p, o, o, l with the controller as it was before the correction was added. This version had no hold phase ($T_s = 0$) and used $\delta = 2.5$ mm.
- Robot, $K = 0.15$: $T_s = 0.8$ s and $\delta = 4$ mm. The run contained six presses; the logger matches letters to presses in order, and only the first three, p, o and l, could be matched to the letter that was sent, so only those three are reported.

Both robot runs used $T_m = 3.0$ s and robot_node at 30 Hz with a 15° step limit. In the $K = 0$ run the logger used the FK of the commanded press angles as the target, which equals $x_k - \delta e_3$ because the IK error was 0.0 mm. The final run, recorded on video, sent the single message "pool" with $K = 0.15$, $\delta = 5.5$ mm and $T_m = 3.0$ s, without the logger.

IV. EXPERIMENTAL RESULTS

A. Forward Kinematics Accuracy

Fig. 5 shows e_{FK} over the 39.6 s run (1980 samples), and Table IV splits it by fingertip speed, with "still" meaning below 5 mm/s. With the arm still, e_{FK} was 0.011 mm on average and at most 0.021 mm, the size of the rounding in the model parameters. While the arm moved, the mean rose to 0.598 mm and the maximum to 1.90 mm. This error comes from the simulator, not from the FK: MuJoCo computes site positions at the start of each 2 ms physics step, before it integrates the joint angles, so `/true_fingertip` belongs to the joint angles of the previous physics step. The fingertip speed times 2 ms follows e_{FK} with a correlation coefficient of 0.97 (Fig. 5). In an offline sweep, recomputing the site

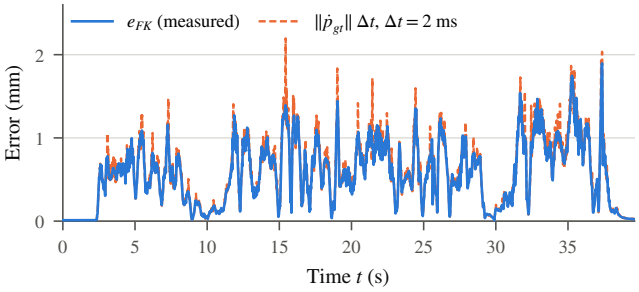


Fig. 5. FK error e_{FK} (11) against the simulator's fingertip while the arm was driven with the leader arm, and the fingertip speed $\|\dot{p}_{gt}\|$ multiplied by the 2 ms physics step. Both are in mm; the FK error while moving matches the one-step reporting lag of the simulator.

TABLE IV

FK ERROR AGAINST SIMULATOR GROUND TRUTH

Condition	Samples	Mean (mm)	95th pct. (mm)	Max (mm)
Arm still	116	0.011	0.011	0.021
Arm moving	1863	0.598	1.196	1.896
All	1980	0.563	1.187	1.896

positions for the current joint angles reduced the error from 1.16 mm to 0.006 mm on average (0.008 mm maximum).

B. Teleoperation and Joint Tracking

With robot_node at its default 10 Hz, `/arm_state` arrived at 10 Hz while leader_node published `/arm_commands` at 30 Hz, so the arm applied one command in three and moved in 100 ms steps. LeRobot also clips every goal to within the step limit of the joint's present position, which at 5° per step capped each joint at 50°/s. With `rate_hz` set to 30 Hz both topics ran at 30.2 Hz, but in a 20 s recording the joints that lift weight stalled: shoulder lift was commanded from $-103.9°$ to $79.5°$ and moved from $-103.4°$ to $-87.1°$, the elbow reached 46.4° of a 99.2° command and wrist flex $-13.6°$ of $-0.7°$, while shoulder pan, which turns about a vertical axis, stayed within 3.4° of its command. Because the clipping is relative to the present position, a lagging joint's servo never sees more than 5° of error, which at a proportional coefficient of 16 did not produce the torque to lift the arm. The step limit was raised to 15° for all later runs. The tracking test was not repeated, but in the typing run without correction shoulder lift was within 2.7° of its command at every press.

C. Key Positions

Table V lists the recorded positions. Evaluating (2) on the recorded `/arm_state` reproduced each recorded `/fk_pose` within 0.3 mm. By (6) the finger was tilted 15.9°, 17.2° and 19.2° from vertical on P, O and L, and 8.8° at home. The home position is 98.7 mm above the mean key-top height, inside the 50–100 mm band the assignment asks for, but 89.4 mm from the centroid of the three keys towards the robot instead of above it; it was kept because IK reaches it from every key with 0.0 mm error. The recorded points are

TABLE V

RECORDED KEY AND HOME POSITIONS IN THE BASE FRAME

Pose	x (mm)	y (mm)	z (mm)
P	347.0	10.1	-10.0
O	364.1	21.9	-7.4
L	354.7	42.0	-9.7
Home	266.0	19.3	89.7

TABLE VI

END-EFFECTOR CONTROL WITH THE INVERSE_KINEMATICS NODE (SIMULATOR)

Target (mm)	Reached (mm)	Error (mm)
(280, 50, 50)	(275.6, 49.1, 44.0)	7.5
(266, 19, 90)	(262.9, 18.8, 84.9)	6.0
(900, 0, 300)	refused	–

20.8 mm (P–O), 22.1 mm (O–L) and 32.7 mm (P–L) apart horizontally, against 19.1, 19.6 and 23.8 mm between key centres at a 19.05 mm key pitch; the 1.7–8.9 mm differences combine where on each cap the fingertip rested with the error of the SO-100 model on the SO-101 arm.

D. Inverse Kinematics and End-Effector Control

Table VI shows the `/ik_target` test in the simulator. The node refused $(900, 0, 300)$ mm and the arm stayed at the previous position. The two reachable targets were missed by 7.5 mm and 6.0 mm, 5.1–6.0 mm of it in z, because the simulated joints also stop short of their commands under gravity; for example, at the pose of Fig. 4 shoulder lift settled at 21.5° for a 20.0° command. The inverse_kinematics node has no correction, so it shows the same steady-state error as keyboard_typer with $K = 0$ in simulation (4.6–5.5 mm, Table VII).

For typing, all twelve waypoints of P, O and L, solved in sequence from the recorded home angles, met (5) with a position error below 0.05 mm. The finger tilt φ was 16–21° at the key waypoints and 0° at home, and consecutive waypoints differed by 11–41° in the joint that moved most. The tilt comes from (4): its norms are not squared, so the cost is not differentiable at zero position error, where L-BFGS-B stops without reducing φ. Squaring both terms (in mm) reduced the tilt to at most 3° in an offline check, but turned the wrist roll by 113–154° on the first move, so the unsquared cost was kept.

E. Typing Accuracy Without and With Feedback

Fig. 6 and Table VII compare the press error without and with the correction. On the robot without correction, all four presses landed 10.3–11.9 mm from the target towards the robot, which on this keyboard is the key to the right: P pressed P and [together, both O presses pressed P, and L pressed the semicolon. 0 of 4 keys were correct. In all four presses the elbow settled 2.1–2.4° from its command in the direction that lowers the fingertip. Linearizing (2) at

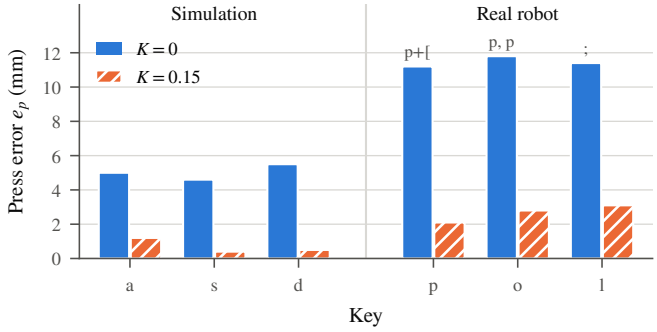


Fig. 6. Press error e_p (12) per key without ($K = 0$) and with ($K = 0.15$) the joint-space correction, in the simulator (keys a, s, d) and on the robot (keys p, o, l). The robot's $K = 0$ bar for o is the mean of two presses (11.9 and 11.7 mm). Labels above the robot's $K = 0$ bars give the key that went down instead.

TABLE VII

KEY-PRESS ERROR e_p

Condition	Mean (mm)	Max (mm)	Keys correct
Sim., $K = 0$	5.0	5.5	3 / 3
Sim., $K = 0.15$	0.7	1.2	3 / 3
Robot, $K = 0$	11.6	11.9	0 / 4
Robot, $K = 0.15$	2.7	3.1	3 / 3
Robot, "pool"	–	–	4 / 4

the recorded key poses, $+1°$ of elbow moves the fingertip 3.6–3.8 mm towards the robot and 2.7–2.9 mm down, so the elbow error alone accounts for 7.5–9.0 mm of the displacement, and wrist-flex errors of up to 1.1° add up to 3.1 mm. The fingertip also went 5.2–7.2 mm deeper than the press target.

With $K = 0.15$, the error fell to 2.1–3.1 mm (mean 2.7 mm, 77 % lower), the fingertip stopped 0.6–0.7 mm above the press target, and 3 of 3 keys were correct. In the simulator the mean fell from 5.0 mm to 0.7 mm (86 % lower), and all three keys registered in both conditions. In two of the robot presses the measured shoulder-lift angle exceeded its corrected command by 5.5–5.7°. Because $|c| \leq 4°$, at least 1.5–1.7° of shoulder-lift error from θ_j^* remained, so a larger c_m could reduce the remaining error. In the final run, the message "pool" produced p, o, o, l on the computer with 4 of 4 keys correct.

The robot conditions also differ in the hold phase and in δ (Section III-E); in the simulator, $K = 0$ included the 0.8 s hold and still missed by 4.6–5.5 mm.

F. Problems and Assumptions

Unlike the robot in the simulation, the real SO-101 is made of 3D-printed plastic, and the gears in the servos have some backlash. This means that there is always going to be some degree of free movement that goes on without being controlled or detected by the servos. With an open-loop control where we just tell the robot to go to certain coordinates, we cannot guarantee that we reach the same point on every attempt. This error is also magnified by the keyboard and robot not being fixed to the desk, so they can move between experiments, which resulted in multiple attempts to achieve success.

The command rate and the step limit of robot_node had to be raised from 10 to 30 Hz and from 5 to 15° (Section IV-B). Before the key mapping, the six arm servos were found to hold the leader arm's calibration offsets. robot_node checks them only with the motors on, so in read-only mode it had reported $-124°$ (shoulder lift) and $-137°$ (elbow) for the folded arm, against $-110.0°$ and $96.9°$ after the robot's calibration was written back. All real-robot data in this report were taken after that.

The kinematic model describes the SO-100 arm, so $p(\theta)$ and e_p on the SO-101 are not ground truth (Section III-E), and the robot comparison is one run per condition, with 4 and 3 presses.

V. CONCLUSION

FK, IK, the end-effector controller and the typing controller run as ROS 2 nodes that drive the simulated and the real LeKiwi without changes. In the simulator the FK matched the true fingertip to 0.011 mm with the arm still, and IK solved every typing waypoint to below 0.05 mm. On the robot, open-loop joint commands were not enough to type: the elbow settled 2.1–2.4° from its command under gravity, the fingertip missed by 11.6 mm on average, and 0 of 4 keys were correct. Integral feedback on the measured joint angles cut the miss to 2.7 mm and made 3 of 3 measured presses and 4 of 4 letters of "pool" correct.

The parts that limited performance were the servos' steady-state error under load, the 5° default step limit, which held shoulder lift to 16.3° of a 183.4° command, the 4° correction limit, which left 1.5–1.7° of shoulder-lift error, the 16–21° finger tilt of the IK solutions, and the time per letter, 15.2 s. Next steps are to measure the fingertip on the robot with a camera or ruler, calibrate the model to the SO-101, repeat the with/without comparison with identical δ and T_s and more presses, raise c_m, and use a squared IK cost with a penalty on wrist-roll motion.

REFERENCES

[1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, U.K.: Cambridge Univ. Press, 2017.

[2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[3] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[4] R. Cadene *et al.*, "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," 2024. [Online]. Available: `https://github.com/huggingface/lerobot`

[5] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[6] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A limited memory algorithm for bound constrained optimization," *SIAM J. Scientific Computing*, vol. 16, no. 5, pp. 1190–1208, 1995.

[7] SIGRobotics-UIUC, "LeKiwi: Low-cost mobile manipulator," 2025. [Online]. Available: `https://github.com/SIGRobotics-UIUC/LeKiwi`